How-To Library

Informatica

Using the DTLREXE Utility to Run the EDMLUCTR Log Print Utility Remotely

© Copyright Informatica LLC 2010, 2021. Informatica

Abstract

You can use the DTLREXE utility on a Linux, UNIX, or Windows system to run the EDMLUCTR Log Print utility on a remote z/OS source system. This article describes how to use this feature to get information about PowerExchange change capture for a z/OS source when you do not have access to the z/OS system.

Supported Versions

- PowerExchange 10.0 or later

Table of Contents

Scenario. . . . 2
Configuration Tasks. . . . 2
Step 1. Allocate a Data Set on z/OS for the EDMLUCTR Control Statements. . . . 3
Step 2. Allocate a Data Set on z/OS for EDMLUCTR Output. . . . 4
Step 3. Create JCL for Executing EDMLUCTR on z/OS. . . . 4
Step 4. Create a File on UNIX That Contains EDMLUCTR Control Statements. . . . 5
Step 5. Create Data Maps for the EDMLUCTR Control Statement Files. . . . 6
Step 6. Create a Workflow That Updates the EDMLUCTR Control Statements on z/OS. . . . 7
Step 7. Create a Script That Calls DTLREXE to Submit the Remote JCL on z/OS. . . . 7
Running EDMLUCTR Remotely to Get CDC Status Information. . . . 8

Scenario

You are a PowerCenter administrator who receives notification that a PowerExchange CDC real-time session with a z/OS source is idle. You need to determine if the CDC session stopped or is waiting for more changes from the source.

You do not have access to the z/OS source system. However, you can configure the DTLREXE utility on UNIX to remotely execute the EDMLUCTR utility on z/OS to get change capture information from PowerExchange Logger log data sets.

In a copy of the EDMLUCTR control statements on UNIX, you specify the relative byte address (RBA) within the log data sets from which you want EDMLUCTR to start reading records. Use a PowerCenter workflow to propagate this RBA to the EDMLUCTR control statement data set on z/OS. Then run a script or batch file on UNIX that calls DTLREXE to remotely submit the EDMLUCTR JCL on z/OS.

DTLREXE returns EDMLUCTR output to a result file on UNIX. This output includes counts of changes captured by change type and registration tag name.

By running DTLREXE and checking the result file multiple times, you can determine if the change counts are increasing over time. If so, PowerExchange is capturing changes.

Configuration Tasks

To implement the scenario, you must complete configuration tasks on the z/OS source system, the UNIX system from which you will run DTLREXE, and the PowerExchange Navigator and PowerCenter client systems.

Coordinate with your z/OS system administrator to perform the configuration tasks on z/OS.

Complete the following tasks:

Step	Task
1	On z/OS, allocate a data set for the EDMLUCTR control statements.
2	On z/OS, allocate a data set for the EDMLUCTR output.
3	On z/OS, create JCL for executing EDMLUCTR.
4	On UNIX, create a file that contains the EDMLUCTR control statements.
5	In the PowerExchange Navigator, create data maps for the EDMLUCTR control statement files on UNIX and z/OS.
6	In PowerCenter, create a workflow that updates the EDMLUCTR control statements on z/OS.
7	On UNIX, create a script that calls DTLREXE to submit the remote JCL on z/OS.

Step 1. Allocate a Data Set on z/OS for the EDMLUCTR Control Statements

You must create a data set on the z/OS source system for the EDMLUCTR control statements.

1. On z/OS, allocate a data set for the EDMLUCTR control statements.

   Set the DCB LRECL keyword to 80.
   This scenario uses a data set name of *your*.H2LLOGP.CARD.

2. In the data set, define the EDMLUCTR control statements.

   This scenario uses the following statements to print summary change capture information for a specific VSAM registration tag name:

```
********************************* Top of Data ***********************************
-SEL  LOGRBA=000000000000  RECORDS=EOF SUMM
[-MASKH2L.V901.SRCV001]
******************************** Bottom of Data *********************************
```

   The following table describes the -SEL parameters:

Parameter	Description
LOGRBA	Specifies the RBA in the log data sets that you want to use as the starting point for the EDMLUCTR utility. Enter a value up to 12 hexadecimal digits in length. In a Post-Log Merge environment, enter a 16-digit hexadecimal time stamp instead. EDMLUCTR starts from the first log record that has an RBA or time stamp that is equal to or greater than the specified RBA or time stamp.
RECORDS	Specifies the number of log records for which EDMLUCTR prints information. For this scenario, enter the option EOF to print all records from the start point that is specified in LOGRBA to the current end of log data. **Note:** If you use the EOF setting in an environment with a high level of change activity, EDMLUCTR might print a large amount of information.
SUMM	Prints total number of change records in the log data sets by change type and registration tag name. Change types are inserts, deletes, and updates.

Include the optional -MASK statement on a second line, without square brackets, to filter the log records from which EDMLUCTR returns output. The mask can be a hexadecimal value, such as a UOW identifier, or a character value, such as a *schema.table_name* or *registration_tag_name*.

- If you enter a hexadecimal value, include a space between -MASK and the value, for example:

  ```
  -MASK hexadecimal_mask
  ```

- If you enter a character value, do not include a space, for example:

  ```
  -MASKcharacter_mask
  ```

Do not include wildcards in the mask. However, you can enter part of a mask value to get multiple matching items.

For more information about EDMLUCTR syntax and parameters, see the *PowerExchange Utilities Guide*.

Step 2. Allocate a Data Set on z/OS for EDMLUCTR Output

You can either permanently allocate a data set on z/OS for the EDMLUCTR output, or allocate the data set as part of the JCL job that executes EDMLUCTR. With either method, set LRECL to 133.

This scenario uses a data set name of *your*.H2LREX.

Step 3. Create JCL for Executing EDMLUCTR on z/OS

On the z/OS system, create a JCL job that uses the DTLREXE utility to execute the EDMLUCTR utility. Use the DTLREXE member in the RUNLIB library as a starting point.

1. Copy the DTLREXE member in the RUNLIB under another name.

   This scenario uses the member name of *hlq.your*.RUNLIB(H2LLOGPR).

2. In the H2LLOGPR member, insert JCL for executing the EDMLUCTR utility and for clearing the output data set from a previous run.

   For the EDMLUCTR EXEC statement, add a LOGPRINT step based on the sample JCL from the LOGPRINT data set in the SAMPLIB library. Before the LOGPRINT step, add an IEFBR14 step that clears a result set from a previous run. Insert these steps where the following comment section occurs in H2LLOGPR:

   ```
   //*-------------------------------------------------------
   //*
   //* <<<<< INSERT DESIRED STEPS HERE >>>>>
   //*
   //*-------------------------------------------------------
   ```

   For this scenario, the entire job appears as follows:

   ```
   //H2LLOGPR JOB (MVS01),'H2LLOGPR',MSGCLASS=X,CLASS=A
   //*
   //*
   //LIBSRCH   JCLLIB ORDER=HLQ.RUNLIB
   //*                                                              *
   //INCS1 INCLUDE MEMBER=GENBULK
   //INCS3 INCLUDE MEMBER=GENCHNG
   //*
   //START    EXEC PGM=DTLNTS,PARM='"%STRTJOB"'
   //STEPLIB  DD DSN=&HLQ..LOADLIB,DISP=SHR
   //*
   //*-------------------------------------------------------
   //* Clears the result set for subsequent EDMLUCTR runs.
   //*-------------------------------------------------------
   //BR14   EXEC PGM=IEFBR14
   //DD1       DD DSN=YOUR.H2LREX,
   //             DISP=(MOD,DELETE,DELETE),
   //             UNIT=3390,SPACE=(CYL,(0,0)),
   //             DCB=(LRECL=133,RECFM=FB,BLKSIZE=26600)
   ```

```
//*
//*-------------------------------------------------------
//* Executes EDMLUCTR and creates the result set.
//*-------------------------------------------------------
//* //LOGPRINT EXEC    PGM=EDMLUCTR
//STEPLIB  DD  DISP=SHR,DSN=HLQ.LOAD          <=== LOAD LIBRARY
//ERDS01   DD  DISP=SHR,DSN=HLQ.YOUR.ERDS01   <=== LOGGER ERDS
//EDMPARMS DD  DISP=SHR,DSN=HLQ.YOUR.USERLIB  <=== EDMSDIR LIBRARY
//EDMMSG   DD  DSN=YOUR.H2LREX,DISP=(NEW,CATLG,DELETE),
//             SPACE=(CYL,(1,1)),UNIT=3390,
//             DCB=(LRECL=133,RECFM=FB,BLKSIZE=26600)
//SYSPRINT DD  SYSOUT=*
//SYSUDUMP DD  SYSOUT=*
//SYSIN    DD DSN=YOUR.H2LLOGP.CARD,DISP=SHR
//*
//         IF ((RC > 4) | (ABEND=TRUE)) THEN
//ENDERR   EXEC PGM=DTLNTS,
//   PARM='"%ENDJOB" C 16'
//STEPLIB  DD DSN=&HLQ..LOADLIB,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SYSUDUMP DD  SYSOUT=*
//SYSIN    DD DSN=YOUR.H2LLOGP.CARD,DISP=SHR
//*
//         IF ((RC > 4) | (ABEND=TRUE)) THEN
//ENDERR   EXEC PGM=DTLNTS,
//   PARM='"%ENDJOB" C 16'
//STEPLIB  DD DSN=&HLQ..LOADLIB,DISP=SHR
//SYSPRINT DD SYSOUT=*
//         ELSE
//ENDOK    EXEC PGM=DTLNTS,
//  PARM='"%ENDJOB"'
//STEPLIB  DD DSN=&HLQ..LOADLIB,DISP=SHR
//SYSPRINT DD SYSOUT=*
//         ENDIF
//*
//
```

3. In the JCL, customize the DDs to point the correct data sets on your z/OS system.

   In particular, ensure that the following DDs point to the correct data sets:

   - In the SYSIN DD, specify the fully qualified name of the data set that contains the EDMLUCTR control cards. This scenario uses the data set name *your*.H2LLOGP.CARD.
   - In the DD1 DD and EDMMSG DD, specify the fully qualified name of the EDMLUCTR output data set. This scenario uses the data set name *your*.H2LREX.

Step 4. Create a File on UNIX That Contains EDMLUCTR Control Statements

On the UNIX system where the DTLREXE script will run, use a text editor to create a .txt file that contains the same EDMLUCTR control statements that you defined on z/OS.

1. In the file, enter the -SEL statement with its parameters on one line.

   This statement must be no longer than 80 characters, for example:

   ```
   -SEL LOGRBA=000000000000 RECORDS=EOF SUMM
   ```

   If you want to filter log records for which EDMLUCTR prints information, include the -MASK statement on the next line.

2. Save the file to a directory of your choice.

   This scenario uses the file name H2LLOGPC.txt.

Step 5. Create Data Maps for the EDMLUCTR Control Statement Files

In the PowerExchange Navigator, create a data map for the EDMLUCTR control statement file on UNIX. Create another data map for the EDMLUCTR control statement data set on z/OS.

You will import the data maps into PowerCenter to create source and target definitions. Then use these definitions in a workflow that propagates the control statement changes that you make on UNIX to the control statement data set on z/OS.

1. In the PowerExchange Navigator, click **Add** > **Data Map**.
2. In the **Name** dialog box, enter the following information:
   - In the **Schema Name** box, enter a user-defined schema name, such as H2L.
   - In the **Access Method** list, select **SEQ**.
   - In the **Data Map Name** box, enter a user-defined data map name, such as LOGP.
   - Clear the **Import Record Definitions** box.
3. Click **Next.**

   The **SEQ Access Method** page appears.
4. In the **File** field, enter the name of the file or data set that contains the EDMLUCTR control statements.
   - On UNIX, the file name is H2LLOGPC.txt.
   - On z/OS, the data set name is *your*.H2LLOGP.CARD.
     This data set name must match the DSN value in the SYSIN DD of the JCL on z/OS.
5. In the **Record Format** box, select **Default**.
6. Click **Finish**.
7. In the **Resource Explorer** pane, right-click the data map name and select **Add Record**.
8. In the **Add Record** dialog box, enter a record name, such as `logp_cards`. Then click **OK**.
9. In the **Record** dialog box, right-click the record and select **Add Field at End**.

   You must add a field to hold the EDMLUCTR control statements.

   The **Add Field** dialog box appears.
10. On the **Name** tab, enter the following information:
    - Enter a field name, such as `logp_cards`.
    - In the **Field Type** list, select CHAR.
    - In the **Length** field, enter 80.
11. Click **OK**.
12. In the **Resource Explorer** pane, right-click the data map name and select **Add Table**.
13. In the **Add Table** dialog box, enter a table name, such as `logp_cards`. Then click **OK**.
14. In the **Resource Explorer** pane, right-click the table and select **Properties**.

    The **Table Properties** dialog box appears.
15. In the **Column Generation** list, select **Refresh with missing columns**. Then click **OK**.
16. Click **File** > **Save**.
17. Click **File** > **Send to Remote Node**.
18. In the **Data Map Remote Node** dialog box, enter the connection information for the remote z/OS system. Then click **OK**.

    In the **Location** list, select the node name for the z/OS system. You must have defined this node name in a NODE statement in the dbmover.cfg file.

For more information about creating data maps, see the *PowerExchange Navigator User Guide*.

Step 6. Create a Workflow That Updates the EDMLUCTR Control Statements on z/OS

In the PowerCenter Designer, create a mapping that maps the EDMLUCTR control statement files. In the Workflow Manager, create a connection, session, and workflow that propagates changes that you make to the EDMLUCTR control statement file on UNIX to the EDMLUCTR control statement data set on z/OS.

When setting up the workflow elements, use the following guidelines:

- Use a PWX NRDB Batch connection.
- Import the data map for the EDMLUCTR control statement file on UNIX to create a source definition.
- Import the data map for the EDMLUCTR control statement data set on z/OS to create a target definition.
- Use these source and target definitions in the mapping.

Tip: You can create one data map for the target control statement data set on z/OS and use it to also create the source definition. In Task Developer, open session properties. On the **Mapping** tab, click **Sources view**. In the **Properties** settings, set the **File Name Override** attribute to point to the control statement file on UNIX.

For more information, see *PowerExchange Interfaces for PowerCenter*.

Step 7. Create a Script That Calls DTLREXE to Submit the Remote JCL on z/OS

On the UNIX system, create a script or batch file that calls the DTLREXE utility to remotely submit the JCL with the EDMLUCTR execution step on z/OS.

For this scenario, use a .bat file that includes the following parameters and syntax:

```
dtlrexe prog=submit loc=zos_node_name fn=\"hlq.RUNLIB(H2LLOGPR)\"
mode=(job,timed) time=60 submittimeout=60 output=\"your.H2LREX\"
result=\"C:\directory\H2LRSLT.txt\"
```

The following table describes these parameters:

Parameter	Description
prog	The DTLREXE program. This value must be submit to submit the remote JCL that includes the EDMLUCTR EXEC step on z/OS.
loc	Your z/OS node name, as defined in the NODE statement of the dbmover file.
fn	The fully qualified name of the member on z/OS that contains the JCL with the EDMLUCTR EXEC step. You must enclose this name with the backslash and escape character, that is \".
mode	A submit mode that is composed of the following values: - **job**. Submits a job. - **timed**. After submitting a job, DTLREXE waits for the job to complete. The wait period is specified in the time parameter.
time	Time, in seconds, to wait for the job to return EDMLUCTR results. This time interval begins when the job is submitted.
submittimeout	Time, in seconds, to wait for the JCL job to start running. If the job does not start within this time interval, the job times out.

Parameter	Description
output	The fully qualified name of the member on z/OS that contains the EDMLUCTR output. You must enclose this name with the backslash and escape character, that is \".
result	The full path and file name of the result file on UNIX to which EDMLUCTR output is sent. You must enclose this entry with a backslash and escape character, that is \".

For more information, see the *PowerExchange Utilities Guide*.

Running EDMLUCTR Remotely to Get CDC Status Information

After you complete configuration, you can run a script on the UNIX system to check the status of change capture activity on the z/OS source when a CDC session appears to be idle.

1. To identify the point in the PowerExchange Logger log data sets from which EDMLUCTR should start scanning records, look for the most recent PWXPC-10081 message in the PowerCenter session log on the UNIX system where the PowerCenter Integration Service runs.

   PWXPC issues a PWXPC-10081 message each time it flushes, or commits, change data to the targets. The flush occurs whenever the **UOW Count** threshold on the PWX CDC connection is reached. The message contains a pair of restart tokens that identifies the PowerExchange position in the change stream at the time of the flush. Use the second restart token to create the RBA from which to start EDMLUCTR.
   The following example message shows the second restart token:

   ```
   PWXPC_10081  [INFO] [CDCDispatcher] raising real-time flush with restart tokens
   [00000602469600000000000000602469600000000000000000], [D1C2E6D340400000060243FA00000000]
   because UOW Count [1] is reached
   ```

2. In the LOGRBA parameter in the EDMLUCTR control statement file on UNIX, enter the middle 12 digits of the second restart token to specify the EDMLUCTR starting point.

   Do not include the first 12 hexadecimal digits from the restart token or the trailing zeroes.
   Based on the previous example message, use the following LOGRBA value:

   ```
   -SEL  LOGRBA=0000060243FA  RECORDS=EOF SUMM
   ```

3. In Workflow Manager, run the workflow.

   The workflow propagates the LOGRBA value that you specified in the EDMLUCTR control statement file on UNIX to the EDMLUCTR control statement data set on z/OS.

4. Run the script on UNIX to have DTLREXE remotely execute the JCL that includes the EDMLUCTR step on z/OS.

   The first time you run the script, manually enter the entire DTLREXE command at the command prompt. This practice makes troubleshooting coding errors easier. After you verify that the script syntax is correct, you can run it by double-clicking the .bat file or by including the command in the workflow as a pre-session command task.

   The EDMLUCTR utility returns output to the H2LRSLT.txt file on UNIX. This file is the one specified in the result parameter of the DTLREXE script.

5. On the UNIX system, view the contents of the H2LRSLT result file.

   Record the number of inserts, replacements (updates), and deletes for each registration tag name.
   The following example message output shows the information to look for:

   ```
            E D M   M E S S A G E   L O G       PAGE 1
   L O G   S T A R
   T
   PWXEDM172502I Log Scan/Print Utility initialization in-progress product level V2.4.05
   05/20/2010
   ```

```
Echo of input from
SYSIN.............
-SEL  LOGRBA=000000000000  RECORDS=EOF SUMM
-MASKH2L.V901.SRCV001
  End of input from
SYSIN.............
PWXEDM172191I EDMLUCTR: LMF will begin transferring data for Log Scan/Print Utility at
X'0000000000000000'
PWXEDM172146I EDMLRDP: LMF now processing YOUR.PRILOG.DS01 for Log Scan/Print Utility
PWXEDM172198I EDMLPOPU: LMF table populate tasks are terminating due to an operator stop or PAC
termination
PWXEDM172195I EDMLUPLU: LMF task for Log Scan/Print Utility is terminating due to an operator stop or PAC
termination
Totals by
Source.............
VSAM.H2L.V901.SRCV001       Isrt= 30 Repl= 0 Dlet= 0 Unk= 0
Open
Uows.....................

L O G   E N
D
```

6. Wait for a time period that you expect is long enough for additional changes to be written to the source. Then run the script again.

   The wait interval depends on the level of change activity in your environment.

7. View the contents of the H2LRSLT result file again.
   - If the number of inserts, replacements, or deletes changed, PowerExchange is actively capturing changes. You might have a problem with the PowerCenter workflow or a TCP/IP failure. For more information, see the PowerCenter session log and PowerExchange detail log.
   - If the number of inserts, replacements, or deletes did not change, PowerExchange change capture is idle and waiting for more changes from the source.

Author

Virginia Pfeifle